AI-Based Traffic Signal Control

A Plain-Language Guide to the Project: What Exists, What's New, and How It Works

---

This document explains the project from the ground up, in simple terms. It covers the problem being solved, the tools and technical terms used (SUMO, DQN, reinforcement learning), what already exists in this research area, exactly what is new about this project, and the step-by-step process being followed from start to finish.

Prepared as a project reference document

Contents

1 1. The Problem: Why Traffic Signals Need to Get Smarter

2 2. Key Terms Explained Simply (SUMO, RL, DQN)

3 3. What Already Exists (Prior Work)

4 4. The Gap: What's Still Missing

5 5. Our Novelty: What This Project Adds

6 6. How the New System Works, Step by Step

7 7. The Project Plan: Three Phases

8 8. Current Status

9 9. Why This Matters

1. The Problem: Why Traffic Signals Need to Get Smarter

Most traffic lights in the real world run on fixed timers or simple rules. They turn green and red on a schedule that does not change, no matter how much traffic is actually there. This wastes time: cars wait at empty intersections, and jams build up at busy ones.

Researchers have been using a branch of Artificial Intelligence called **Reinforcement Learning (RL)** to build traffic signal controllers that learn, from experience, when to switch lights based on real traffic conditions. These AI controllers often perform much better than fixed-timer systems in tests.

However, there is a serious catch: these AI controllers are trained and tested in a stable, predictable environment. In the real world, conditions change unexpectedly — a sensor might glitch, traffic patterns shift after a nearby event, or a signal might not respond the way it's supposed to. When this happens, most AI controllers do not know that anything has changed, and their performance can quietly get worse without anyone noticing. This project is about fixing that blind spot.

2. Key Terms Explained Simply

2.1 Reinforcement Learning (RL)

Reinforcement Learning is a way of training software to make decisions by trial and error, similar to how a person learns to play a video game. The software (called an **agent**) takes an action, sees what happens, and gets a **reward** (a score) that tells it whether that action was good or bad. Over many attempts, it learns which actions lead to the best long-term results.

In traffic signal control, the "agent" is the traffic light controller. Its "action" is choosing which direction gets the green light. Its "reward" is usually based on things like how little cars had to wait, or how smoothly traffic moved.

2.2 DQN — Deep Q-Network

DQN stands for **Deep Q-Network**. It is one of the most well-known methods for doing Reinforcement Learning. Here is how to think about it in simple terms:

- The traffic controller constantly looks at the current situation at the intersection — for example, how many cars are waiting on each road, and which direction currently has the green light. This snapshot of the situation is called the **observation** or **state**.
- For every possible action it could take (e.g. "keep the current green" or "switch to the next phase"), DQN estimates a number called a **Q-value**, which represents how good that action is expected to be in the long run, not just right now.
- "Deep" refers to the fact that a **deep neural network** (a layered mathematical model loosely inspired by the brain) is used to estimate these Q-values, because the number of possible traffic situations is far too large to write down by hand.
- The controller simply picks the action with the highest estimated Q-value at each step, and updates its network over time based on how things actually turned out.

In short: **DQN is the "brain"** that decides which direction gets the green light, based on live traffic conditions, and it keeps improving that decision-making from experience.

2.3 SUMO — Simulation of Urban Mobility

Training an AI controller directly on real traffic lights would be slow, expensive, and risky (a bad decision could actually cause real jams or accidents). Instead, researchers use a traffic simulator.

SUMO (Simulation of Urban Mobility) is free, widely-used software that creates a realistic virtual city: roads, intersections, traffic lights, and moving vehicles, all following realistic driving behavior. It lets researchers:

- Build a virtual road network that mimics a real or synthetic intersection layout.
- Generate traffic (cars arriving, turning, queueing) under different conditions.
- Run the same scenario thousands of times quickly, which would be impossible in real life.
- Plug in an AI controller and let it control the simulated traffic lights, then measure results like average waiting time or queue length.

In this project, SUMO is used to generate **synthetic data** — realistic but artificially generated traffic scenarios — which is what the DQN controller is trained and tested on.

2.4 Distribution Shift

An AI model is trained on a certain "distribution" of situations — a certain typical pattern of what traffic normally looks like. A **distribution shift** happens when the real conditions during actual use start to look meaningfully different from what the model was trained on. This project focuses on four kinds of shift, explained here in plain terms:

Type of Shift	What It Means in Simple Terms
Observation shift	The information the controller "sees" becomes unreliable or different than expected — for example, a ser
Transition shift	The way traffic itself behaves changes — for example, driving patterns, arrival rates, or road usage shift
Action shift	The controller's commands do not translate into the expected real-world effect — for example, a signal re
Reward shift	The way "success" is measured or experienced changes — for example, what used to count as good per

If a controller doesn't realize one of these shifts has happened, it keeps making decisions as if nothing changed, and its performance can degrade silently. That silent failure is exactly the risk this project is designed to catch and correct.

3. What Already Exists (Prior Work)

This project builds on, and is positioned against, a set of existing traffic-control approaches. Here is what each of them does, described simply:

Existing Approach	What It Is	Its Limitation
Fixed-Time Controller	A traditional traffic light that switches on a fixed, pre-set timer, regardless of actual traffic	Cannot adapt at all to real-time conditions, focused mainly as a ba
Max Pressure Controller	A rule-based (non-learning) method that always gives green to the direction with the most "pressure" (cars w	Reacts only to current traffic, but with no learning ability or
Standard DQN-based RL controller (e.g. Vidali's baseline)	A learning-based controller, as described above, trained purely on the conditions it was trained on	Performs well only in the conditions it was trained on, but has no
RobustLight	A research method aimed at making RL traffic controllers more robust to certain disturbances	Improves robustness in specific, narrow ways, but does not offe
Sim2Signal	A research method focused on transferring a model trained in simulation to the real world	Addresses the sim-to-real gap, but does not provide wo
RESCO	A widely-used benchmark suite / framework for comparing different RL traffic signal control methods under	Useful for fair comparisons, but is a benchmarking

In short: some existing methods are simple and predictable but cannot adapt (Fixed-Time, Max Pressure); others are adaptive and high-performing but fragile under changing conditions (standard DQN); and others improve robustness or transferability in specific, narrow ways (RobustLight, Sim2Signal) without offering a complete detect → diagnose → respond framework.

4. The Gap: What's Still Missing

Across all of the existing work above, no method currently does all of the following together:

1. **Continuously monitor** a live RL traffic controller for signs that something has changed from normal conditions.
2. **Correctly identify which of the four types of shift** is occurring — observation, transition, action, or reward — rather than just flagging "something seems off."
3. **Actively respond** to the specific shift detected, adjusting the controller's behavior accordingly, rather than leaving it to degrade or requiring a human to intervene.

> This three-part gap — detect, diagnose, and mitigate, specifically for RL-based traffic signal control — is the concrete research gap this project targets.

5. Our Novelty: What This Project Adds

The core contribution of this project is a standard DQN-based traffic light controller, augmented with **three new modules** that work together to close the gap above:

Module	Simple Explanation of What It Does
1. Shift Monitor	Continuously watches the controller's incoming data and behavior while it runs, looking for statistical s
2. Diagnosis Module	Once the Shift Monitor raises an alert, this module figures out which specific type of shift is happening

3. Mitigation Module	Based on the diagnosis, this module actively adjusts the controller's behavior to compensate for the s

Put together, these three modules form a closed loop: **Detect** → **Diagnose** → **Mitigate**. This is what distinguishes this project from the standard DQN baseline (which does none of this) and from RobustLight, Sim2Signal, and RESCO (which each address only part of the picture, and not for this specific combination of shift types).

6. How the New System Works, Step by Step

Here is the full working process of the system, from a single moment of traffic, described as a simple sequence anyone can follow:

Step 1 — Observe

The system takes a snapshot of the intersection: how many cars are waiting in each direction, and which light is currently green. This is the same "observation" step a normal DQN controller would do.

Step 2 — Decide

The DQN "brain" looks at this snapshot and decides which direction should get the green light next, based on everything it has learned so far.

Step 3 — Monitor

At the same time, the Shift Monitor checks this observation, the action taken, and the reward received against what "normal" has looked like during training. If everything matches expectations, the system carries on as usual.

Step 4 — Diagnose (only if a shift is flagged)

If the Shift Monitor detects something unusual, the Diagnosis Module examines the pattern of the anomaly and classifies it as one of the four shift types: observation, transition, action, or reward.

Step 5 — Mitigate (only if a shift is flagged)

The Mitigation Module then takes a corrective action suited to that specific shift type, so the controller can keep performing reasonably well instead of continuing to make decisions as if nothing had changed.

Step 6 — Repeat

This entire cycle repeats continuously, every time the controller needs to make a decision, so the system is always watching, and always ready to respond.

7. The Project Plan: Three Phases

The project is carried out in three phases, each building on the last. This structure makes sure the basic controller is proven to work fairly before the new shift-handling modules are added and tested.

Phase 1 — Baseline Comparison

Before introducing anything new, the project first establishes a fair, trustworthy baseline. This means training and testing three controllers under normal (non-shifted) conditions, using traffic scenarios generated in SUMO:

- **DQN agent** — trained across multiple random seeds (independent training runs) to make sure results are not a fluke of one lucky run.
- **Fixed-Time controller** — the traditional, non-adaptive method, used as a lower-bar comparison.
- **Max Pressure controller** — a strong rule-based method, used as a tougher comparison.

Exit criteria for Phase 1: all controllers are evaluated fairly, using the same conditions and the same evaluation method, across multiple seeds, with model checkpoints (saved versions of the trained DQN) kept so results can be verified and reproduced later.

Phase 2 — Shift Detection and Classification

Once the baseline is solid, the project introduces distribution shifts into the simulation (observation, transition, action, and reward shifts, one at a time and combined) and builds and tests the **Shift Monitor** and **Diagnosis Module** against them. The goal here is to prove the system can reliably notice that something has changed, and correctly identify which of the four types it is.

Phase 3 — Mitigation and Full System Evaluation

Finally, the **Mitigation Module** is added so the full pipeline (Monitor → Diagnose → Mitigate) is active. The complete system is then tested against the same shifted conditions used in Phase 2, and compared to: (a) a plain DQN controller with no shift-handling ability, and (b) the Fixed-Time and Max Pressure baselines, to measure how much more resilient the new system is when conditions change unexpectedly.

8. Current Status

As of now, the project is in the middle of **Phase 1**. Along the way, a few important issues were caught and are being corrected before moving forward, which is a normal and healthy part of doing rigorous research:

- An earlier training run was found to have never saved its model checkpoints, meaning the trained DQN could not actually be reloaded and fairly re-tested later. Training is being re-run with checkpoint saving properly enabled.
- All five random seeds (independent training runs) must fully complete before Phase 1 is considered done; a premature "complete" claim, based on only some seeds finishing, was caught and corrected.
- An inconsistency was found in how the DQN's results and the baselines' results were being measured, which is being resolved so that all three controllers are compared on exactly equal terms.

Independent, file-based verification (checking what result files actually exist on disk, rather than trusting summary claims) is being used throughout to make sure Phase 1 is only marked complete when it genuinely is.

9. Why This Matters

Traffic signal control is a real, high-stakes system: it affects commute times, fuel use, emissions, and safety, every single day. AI controllers offer major efficiency gains over traditional timers, but only if they can be trusted to keep working well when real-world conditions inevitably drift from what they were trained on. This project's contribution — a controller that can notice change, understand what kind of change it is, and adjust its own behavior in response — is a concrete step toward making AI-based traffic control robust enough for real deployment, not just strong performance in a clean lab simulation.

End of document.